Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Chaithra B C **1BM25EC406**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

INDEX

Sl. No.	Contents	Page No.
	Introduction	1
1	Arithmetic Logic Unit (ALU)	2-3
2	Four-Bit Comparator	4-5
3	Four-Bit Parity Generator and Checker	6
4	Four-Bit Barrel Shifter	7
5	Timer Module	8
6	Four-Bit Ring Counter	9
7	Johnson Counter	10
8	Synchronous and Asynchronous Counters	11-12
9	Four-Bit Shift Register	13
10	Divide-by-N Counter	14
11	Mealy and Moore Finite State Machines (FSMs)	15-17
12	Random Sequence Generator (4-Bit LFSR)	18

INTRODUCTION

EDA Playground is an online platform used to write, simulate, and verify digital circuits using Hardware Description Languages (HDLs) such as Verilog and VHDL. It provides a convenient environment for designing and testing digital circuits without requiring any additional software installation.

In this AAT, EDA Playground is used to simulate and verify various digital circuits designed using Verilog HDL. The Verilog code describes the functionality of each circuit, while the testbench provides the required input signals and checks the circuit's response. The simulation results are observed using the output and waveform viewer to verify the correct working of the designed circuits.

The experiments include combinational circuits, such as an Arithmetic Logic Unit (ALU), comparator, parity generator, and barrel shifter, as well as sequential circuits, such as counters, shift registers, timers, and finite state machines (FSMs).

Thus, EDA Playground helps in understanding the operation of digital circuits, verifying their functionality, and analysing their outputs through simulation before hardware implementation.

1. ARITHMETIC LOGIC UNIT (ALU)

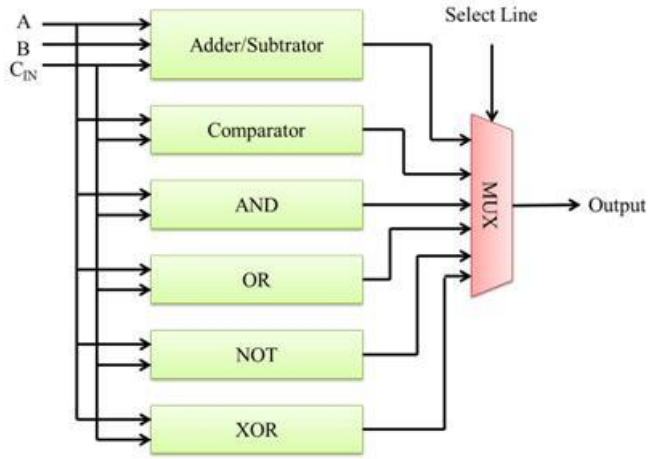
Aim: To design and implement a 4-bit Arithmetic Logic Unit (ALU) using Verilog HDL and verify its operation using a testbench.

Theory: A 4-bit Arithmetic Logic Unit (ALU) is a combinational digital circuit that performs arithmetic and logical operations on two 4-bit inputs. The operation-select lines determine which function is performed. Typical functions include addition, subtraction, AND, OR, XOR, NOT, left shift, and passing input A to the output.

Working: The inputs A and B and the select signal are applied to the ALU. Depending on the select value, the circuit performs the chosen operation and places the result on the output. For arithmetic operations, the carry-out can indicate a carry beyond the 4-bit result. In EDA Playground, the testbench applies different input combinations and operation-select values; the output waveform is checked against the expected result.

Operations: Addition, subtraction, AND, OR, XOR, NOT, left shift, and pass input A.

Design



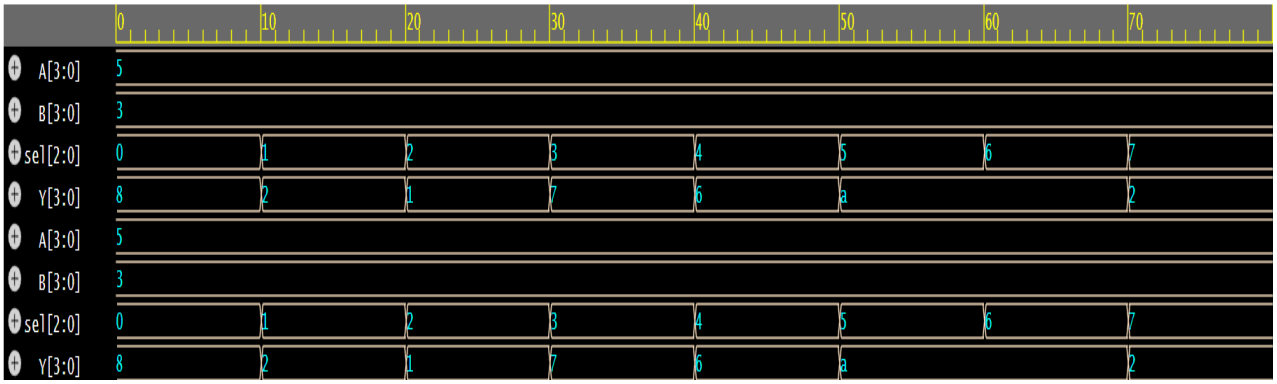
Code

```v
// Testbench
module tb_ALU;
reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] Y;
ALU uut (
    .A(A),
    .B(B),

    .sel(sel),
    .Y(Y)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    A = 4'b0101;
    B = 4'b0011;
    sel = 3'b000; #10;
    sel = 3'b001; #10;
    sel = 3'b010; #10;
    sel = 3'b011; #10;
    sel = 3'b100; #10;
    sel = 3'b101; #10;
    sel = 3'b110; #10;
    sel = 3'b111; #10;
    $finish;
end
initial begin
    $monitor("Time=%0t A=%b B=%b sel=%b Y=%b",
             $time, A, B, sel, Y);
end
endmodule
```

```v
// Design: 4-bit ALU
module ALU(
    input [3:0] A, B,
    input [2:0] sel,
    output reg [3:0] Y
);
always @(*) begin
    case (sel)
        3'b000: Y = A + B;
        3'b001: Y = A - B;
        3'b010: Y = A & B;
        3'b011: Y = A | B;
        3'b100: Y = A ^ B;
        3'b101: Y = ~A;
        3'b110: Y = A << 1;
        3'b111: Y = A >> 1;
        default: Y = 4'b0000;
    endcase
end
endmodule
```

Output



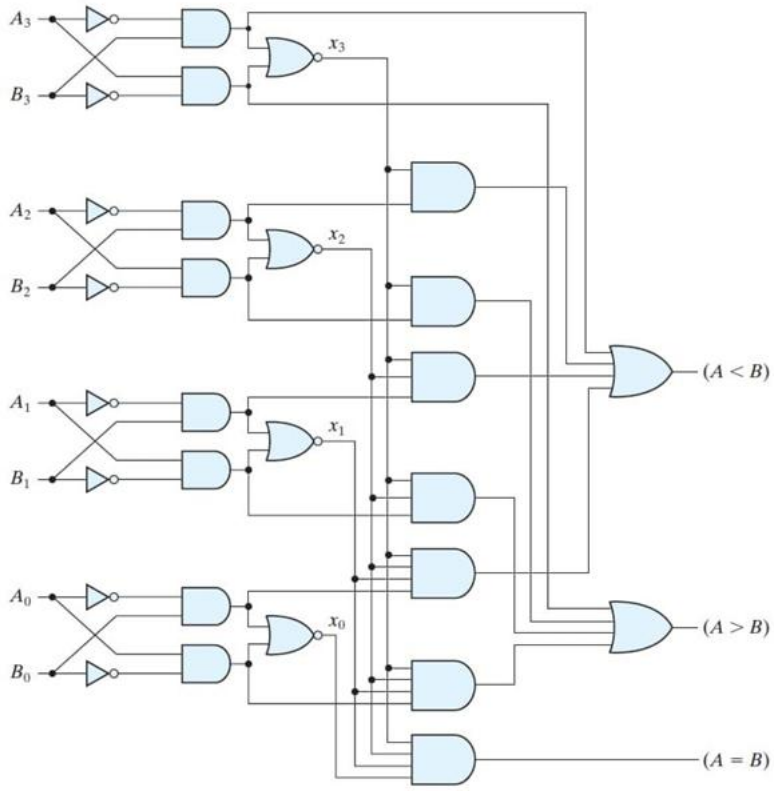
2. FOUR-BIT COMPARATOR

Aim: To design and implement a 4-bit comparator using Verilog HDL to compare two binary numbers.

Theory: A 4-bit comparator is a combinational circuit that compares two unsigned 4-bit binary numbers, A and B. It produces three mutually exclusive outputs: A greater than B, A less than B, or A equal to B.

Working: The circuit examines the values of A and B and activates the output corresponding to their relationship. If A is larger, A>B becomes 1; if B is larger, A<B becomes 1; if both numbers are the same, A==B becomes 1.

Logic diagram



Code

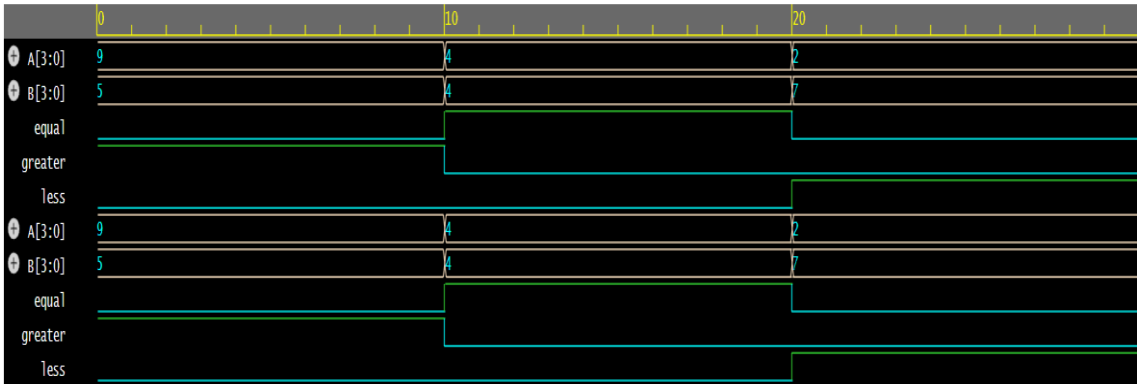
```
// Testbench
module tb_comparator;
reg [3:0] A, B;
wire greater, equal, less;

comparator uut (
    .A(A),
    .B(B),
    .greater(greater),
    .equal(equal),
    .less(less)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    A = 4'd9; B = 4'd5; #10;
    A = 4'd4; B = 4'd4; #10;
    A = 4'd2; B = 4'd7; #10;
    $finish;
end
initial begin
    $monitor("Time=%0t A=%d B=%d Greater=%b Equal=%b
Less=%b",
             $time, A, B, greater, equal, less);
end
endmodule
```

```
// Design: 4-bit Comparator
module comparator(
    input [3:0] A, B,
    output reg greater,
    output reg equal,
    output reg less
);
always @(*) begin
    greater = 0;
    equal = 0;
    less = 0;
    if (A > B)
        greater = 1;
    else if (A == B)
        equal = 1;
    else

        less = 1;
end
endmodule
```

Output



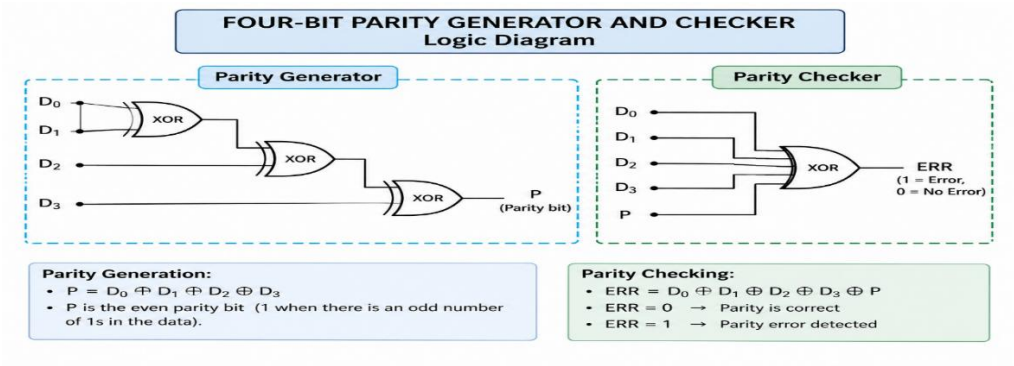
3. FOUR-BIT PARITY GENERATOR AND CHECKER

Aim: To design a 4-bit even-parity generator and checker using Verilog HDL.

Theory: An even-parity generator adds a parity bit to data so that the total number of 1s becomes even. A parity checker checks the received data and parity bit to identify a possible single-bit error.

Working: The XOR of the four input bits determines the even-parity bit. The checker XORs the data bits with the received parity bit; a 0 result indicates even parity, while a 1 result indicates a parity mismatch.

Logic diagram

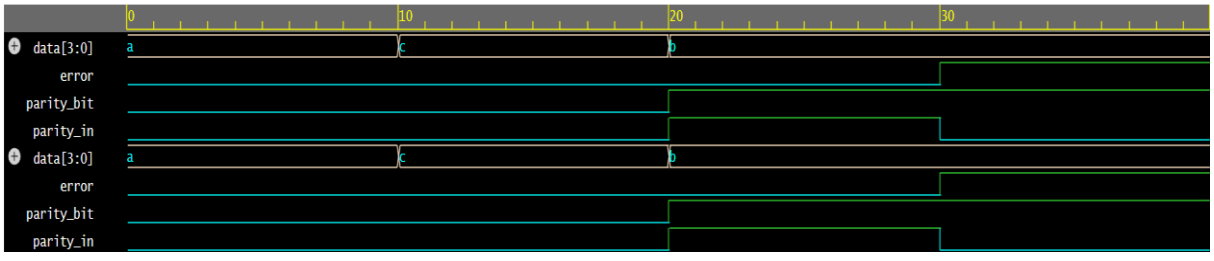


Code

```
module tb_parity_generator_checker;
reg [3:0] data;
reg parity_in;
wire parity_bit, error;
parity_generator_checker uut (
    .data(data),
    .parity_in(parity_in),
    .parity_bit(parity_bit),
    .error(error)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    data = 4'b1010;
    parity_in = 1'b0;
    #10;
    data = 4'b1100;
    parity_in = 1'b0;
    #10;
    data = 4'b1011;
    parity_in = 1'b1;
    #10;
    data = 4'b1011;
    parity_in = 1'b0;
    #10;
    $finish;
end
initial begin
    $monitor("Time=%0t Data=%b ParityBit=%b ParityIn=%b
Error=%b",
             $time, data, parity_bit, parity_in, error);
end
endmodule
```

```
// Design: 4-bit Parity Generator and Checker
module parity_generator_checker(
    input [3:0] data,
    input parity_in,
    output parity_bit,
    output error
);
assign parity_bit = ^data;
assign error = (^data) ^ parity_in;
endmodule
```

Output



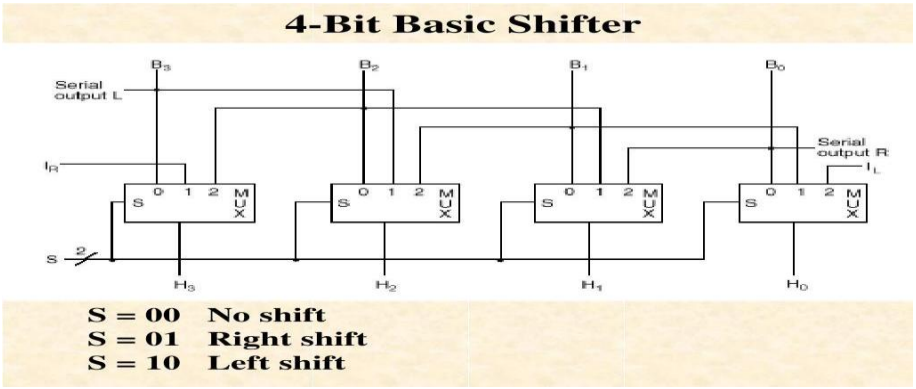
4. FOUR-BIT BARREL SHIFTER

Aim: To design and implement a 4-bit barrel shifter using Verilog HDL for logical left and right shifting.

Theory: A 4-bit barrel shifter is a combinational circuit that shifts a 4-bit input left or right by a selected number of bit positions. Vacated positions are filled with zeros during logical shifting.

Working: The input data, shift amount, and direction control are applied. The circuit shifts the bits by the selected amount and presents the shifted value at the output. Bits shifted beyond the 4-bit range are discarded.

Logic diagram



Code

```
// Testbench
module tb_barrel_shifter;
reg [3:0] data;
reg [1:0] shift;

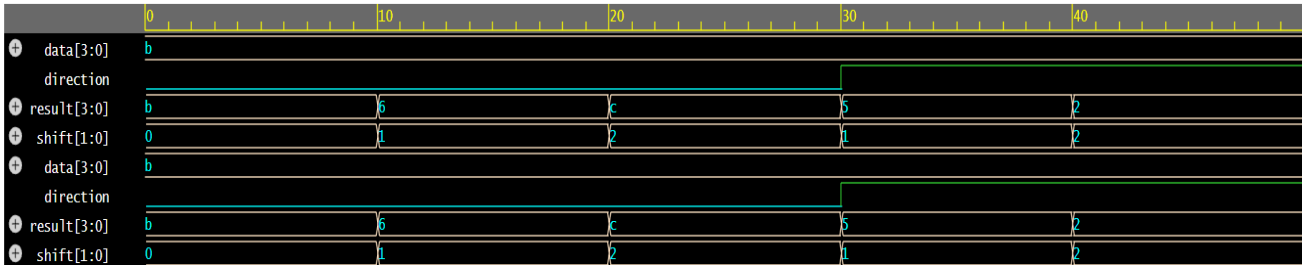

reg direction;
wire [3:0] result;

barrel_shifter uut (
    .data(data),
    .shift(shift),
    .direction(direction),
    .result(result)
);
initial begin

    $dumpfile("dump.vcd");
    $dumpvars;
    data = 4'b1011;
    direction = 0; shift = 0; #10;
    direction = 0; shift = 1; #10;
    direction = 0; shift = 2; #10;
    direction = 1; shift = 1; #10;
    direction = 1; shift = 2; #10;
    $finish;
end
initial begin
    $monitor("Time=%0t Data=%b Direction=%b Shift=%d
Result=%b",
             $time, data, direction, shift, result);
end
endmodule
```

```
// Design: 4-bit Barrel Shifter
module barrel_shifter(
    input [3:0] data,
    input [1:0] shift,
    input direction,
    output reg [3:0] result
);
always @(*) begin
    if (direction == 1'b0)
        result = data << shift;
    else
        result = data >> shift;
end
endmodule
```

Output

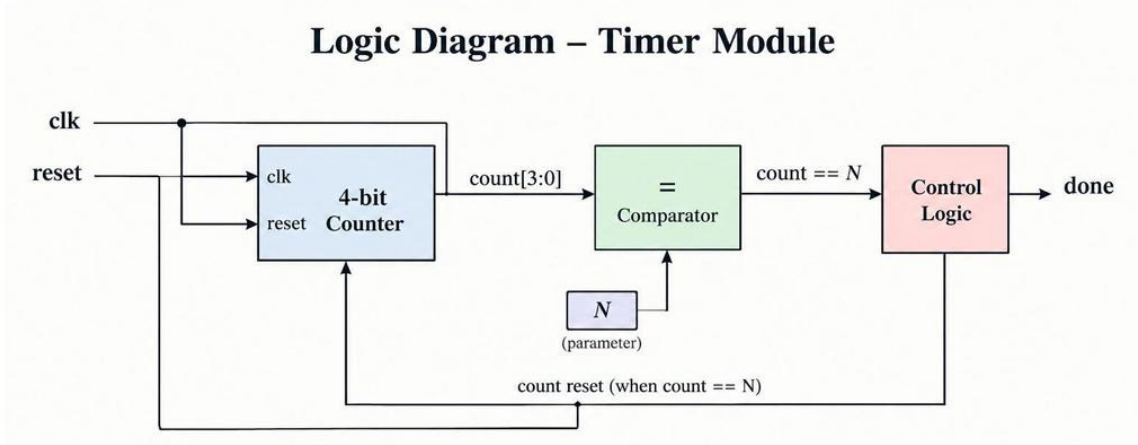


5. TIMER MODULE

Aim: To design a timer module using Verilog HDL that generates a periodic pulse after a specified number of clock cycles.

Theory: A timer module counts clock cycles and generates an output pulse when a programmed count is reached. It is commonly used to create delays or periodic control signals in digital systems.

Working: On each active clock edge, the counter increments. When it reaches the specified terminal count, the output pulse is generated and the counter is reset or restarted according to the design.



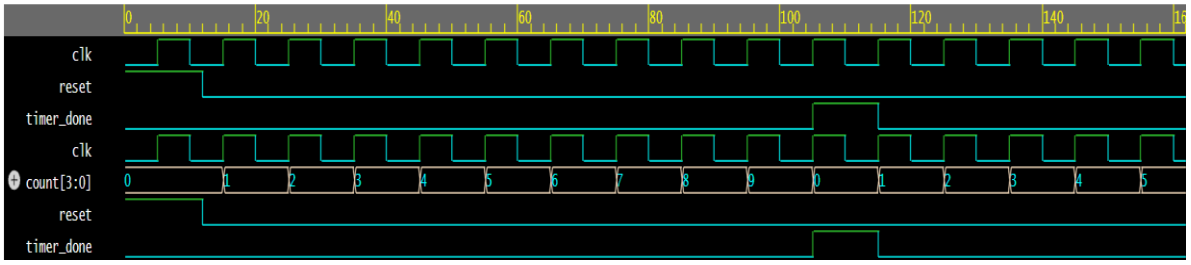
Code

```
// Testbench
module tb_timer_module;
reg clk, reset;
wire timer_done;
timer_module uut (
    .clk(clk),

    .reset(reset),
    .timer_done(timer_done)
);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    clk = 0;
    reset = 1;
    #12 reset = 0;
    #150;
    $finish;
end
initial begin
    $monitor("Time=%0t Reset=%b TimerDone=%b",
             $time, reset, timer_done);
end
endmodule
```

```
// Design: Timer Module
module timer_module(
    input clk,
    input reset,
    output reg timer_done
);
reg [3:0] count;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        count <= 0;
        timer_done <= 0;
    end
    else begin
        if (count == 4'd9) begin
            count <= 0;
            timer_done <= 1;
        end
        else begin
            count <= count + 1'b1;
            timer_done <= 0;
        end
    end
end
endmodule
```

Output



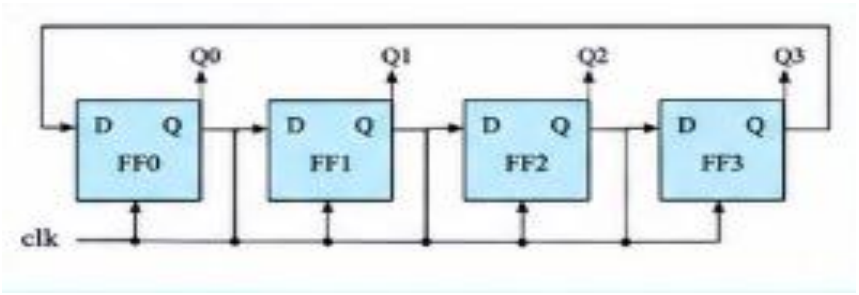
6. FOUR-BIT RING COUNTER

Aim: To design and implement a 4-bit ring counter using Verilog HDL.

Theory: A 4-bit ring counter is a sequential circuit made from four flip-flops connected as a shift register, with the last output fed back to the first input. It typically circulates a single 1 through four states.

Working: After initialization to a one-hot state such as 0001, each clock pulse shifts the 1 to the next flip-flop. After four clock pulses, the pattern returns to its starting state.

Logic diagram



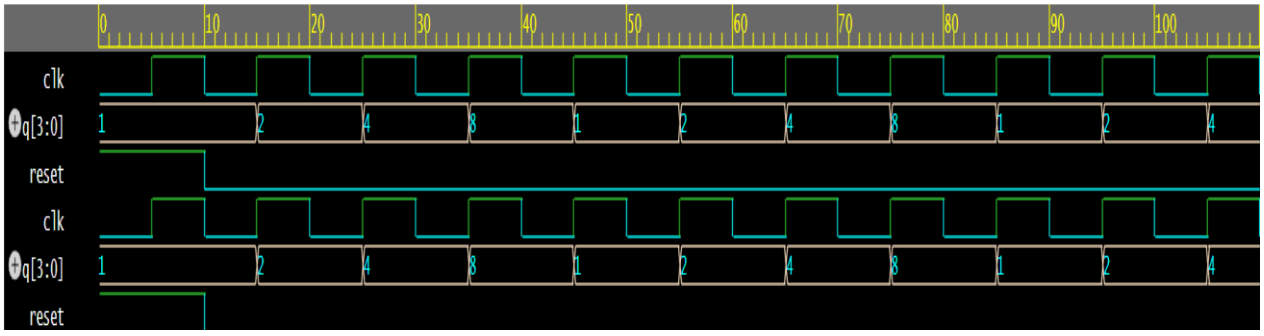
Code

```verilog
// Testbench
module tb_ring_counter;
reg clk, reset;
wire [3:0] q;
ring_counter uut (
    .clk(clk),
    .reset(reset),
    .q(q)
);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");

    $dumpvars;
    clk = 0;
    reset = 1;
    #10 reset = 0;
    #100;
    $finish;
end
initial begin
    $monitor("Time=%0t Reset=%b Q=%b",
             $time, reset, q);
end
endmodule
```

```verilog
// Design: 4-bit Ring Counter
module ring_counter(
    input clk,
    input reset,
    output reg [3:0] q
);
always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 4'b0001;
    else
        q <= {q[2:0], q[3]};
end
endmodule
```

Output



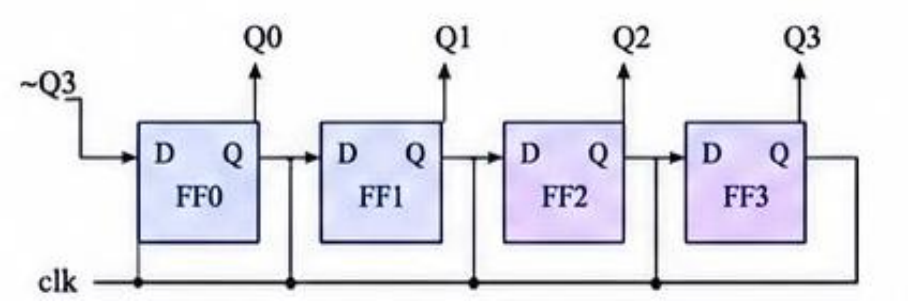
7. Johnson Counter

Aim: To design and implement a 4-bit Johnson counter using Verilog HDL.

Theory: A 4-bit Johnson counter is a shift-register counter in which the complemented output of the last flip-flop is fed back to the input of the first. A 4-bit Johnson counter can produce up to eight distinct states.

Working: At every active clock edge, the bits shift by one position and the inverted last bit enters the first stage. The sequence progresses through the Johnson states and repeats.

Block diagram



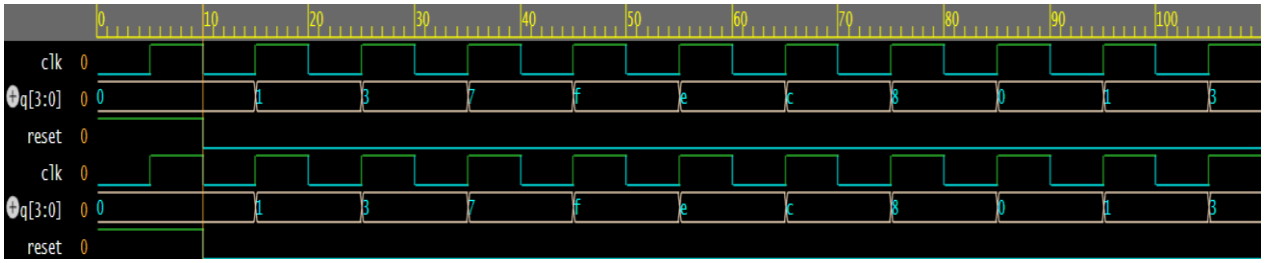
Code

```
// Testbench
module tb_johnson_counter;
reg clk, reset;
wire [3:0] q;
johnson_counter uut (
    .clk(clk),
    .reset(reset),
    .q(q)
);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");

    $dumpvars;
    clk = 0;
    reset = 1;
    #10 reset = 0;
    #100;
    $finish;
end
initial begin
    $monitor("Time=%0t reset=%b q=%b", $time, reset, q);
end
endmodule
```

```
// Design: 4-bit Johnson Counter
module johnson_counter(
    input clk,
    input reset,
    output reg [3:0] q
);
always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 4'b0000;
    else
        q <= {q[2:0], ~q[3]};
end
endmodule
```

Output



8. Synchronous and Asynchronous Counters

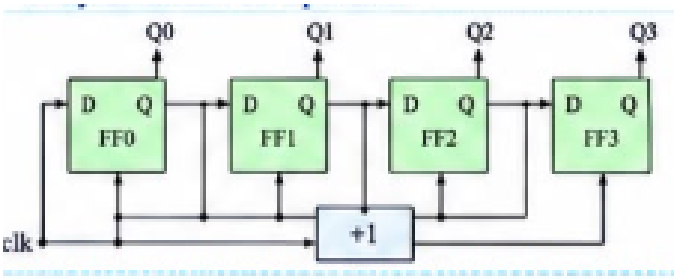
A. Synchronous 4-Bit Up-Counter

Aim: To design a 4-bit synchronous up-counter using Verilog HDL.

Theory: A synchronous 4-bit up-counter is a sequential circuit in which all four flip-flops receive the same clock signal. It counts upward in binary from 0000 to 1111 and then wraps around.

Working: On each active clock edge, all counter bits update together to represent the next binary value. After 1111, the next count returns to 0000; any reset or enable behaves as specified in the Verilog design.

Block diagram



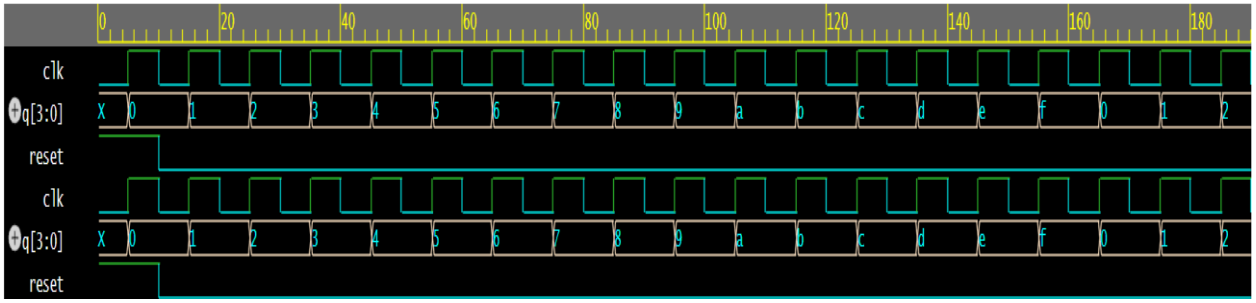
Code

```verilog
// Testbench
module tb_synchronous_counter;
reg clk, reset;
wire [3:0] q;
synchronous_counter uut (
    .clk(clk),
    .reset(reset),
    .q(q)
);
always #5 clk = ~clk;
initial begin

    $dumpfile("dump.vcd");
    $dumpvars;
    clk = 0;
    reset = 1;
    #10 reset = 0;
    #180;
    $finish;
end
initial begin
    $monitor("Time=%0t reset=%b q=%d", $time, reset, q);
end
endmodule
```

```verilog
// Design: Synchronous Counter
module synchronous_counter(
    input clk,
    input reset,
    output reg [3:0] q
);
always @(posedge clk) begin
    if (reset)
        q <= 4'b0000;
    else
        q <= q + 1'b1;
end
endmodule
```

Output



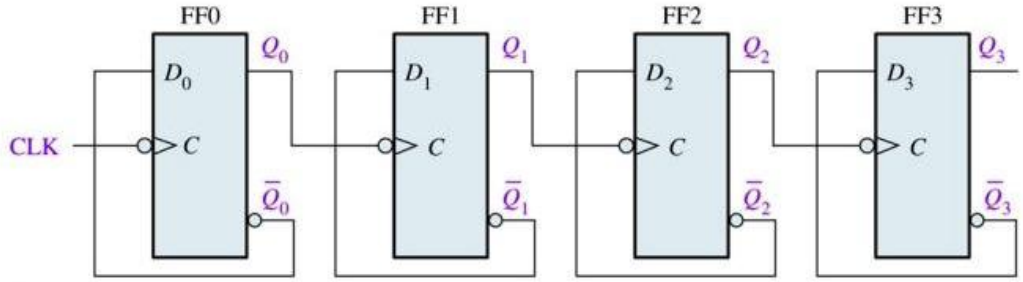
B. Asynchronous 4-Bit Ripple Up-Counter

Aim: To design a 4-bit asynchronous up-counter using Verilog HDL.

Theory: An asynchronous 4-bit ripple up-counter uses flip-flops that do not all share the same clock. The output of one stage clocks the next stage, so changes ripple through the counter.

Working: The first flip-flop responds to the external clock, and its output drives the next stage. Subsequent stages toggle as the transitions propagate, producing a binary count from 0000 through 1111.

Block diagram



Code

```
// Testbench
module tb_asynchronous_counter;
reg clk, reset;
wire [3:0] q;
asynchronous_counter uut (
    .clk(clk),
    .reset(reset),
    .q(q)
);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    clk = 0;
    reset = 1;
    #12 reset = 0;
    #200;
    $finish;
end
initial begin
    $monitor("Time=%0t reset=%b q=%d", $time, reset, q);

end
endmodule

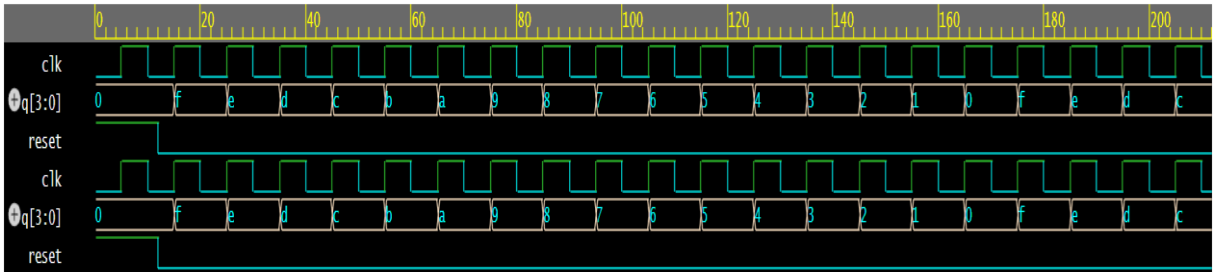
```

```
// Design: Asynchronous Counter
module asynchronous_counter(
    input clk,
    input reset,
    output reg [3:0] q
);
always @(posedge clk or posedge reset) begin
    if (reset)
        q[0] <= 1'b0;
    else
        q[0] <= ~q[0];

end
always @(posedge q[0] or posedge reset) begin
    if (reset)
        q[1] <= 1'b0;
    else
        q[1] <= ~q[1];

end
always @(posedge q[1] or posedge reset) begin
    if (reset)
        q[2] <= 1'b0;
    else
        q[2] <= ~q[2];
end
always @(posedge q[2] or posedge reset) begin
    if (reset)
        q[3] <= 1'b0;
    else
        q[3] <= ~q[3];
end
endmodule
```

Output



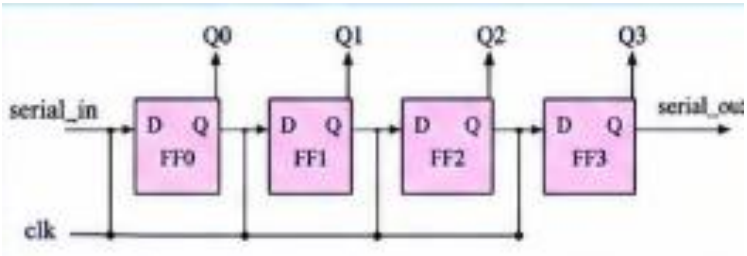
9. Four-Bit Shift Register

Aim: To design a 4-bit serial-in, serial-out (SISO) shift register using Verilog HDL.

Theory: A 4-bit serial-in, serial-out (SISO) shift register stores data in four flip-flops and transfers one bit at a time. It accepts serial input and provides serial output after the data shifts through the stages.

Working: At each active clock edge, the serial input enters the first stage and the existing bits move to the next stages. The bit at the final stage appears at the serial output after the required clock cycles.

Block digram



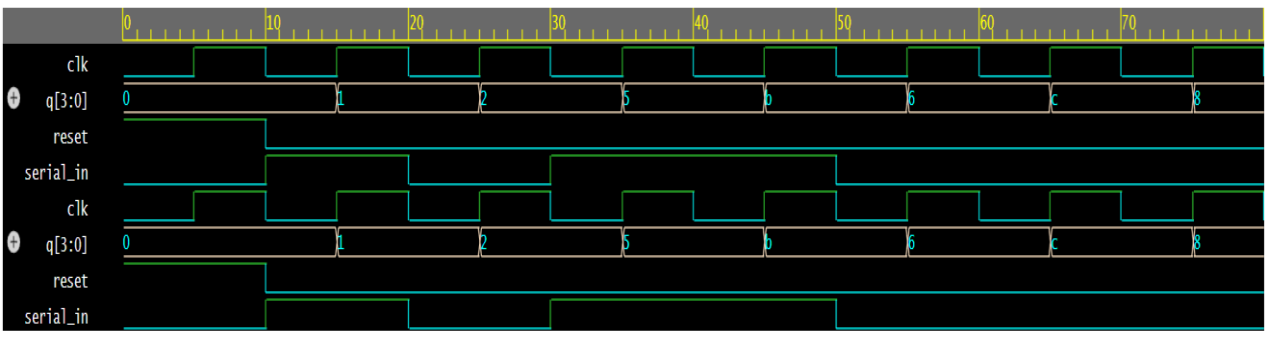
Code

```
// Testbench
module tb_shift_register;
reg clk, reset, serial_in;
wire [3:0] q;
shift_register uut (
    .clk(clk),
    .reset(reset),
    .serial_in(serial_in),
    .q(q)
);
always #5 clk = ~clk;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    clk = 0;
    reset = 1;
    serial_in = 0;
    #10 reset = 0;
    serial_in = 1; #10;
    serial_in = 0; #10;
    serial_in = 1; #10;
    serial_in = 1; #10;
    serial_in = 0; #10;
    #20;
    $finish;
end
initial begin
    $monitor("Time=%0t serial_in=%b q=%b",
             $time, serial_in, q);
end
endmodule
```

```
// Design: 4-bit Shift Register
module shift_register(
    input clk,
    input reset,
    input serial_in,
    output reg [3:0] q
);
always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 4'b0000;
    else
        q <= {q[2:0], serial_in};
end
endmodule
```

Output



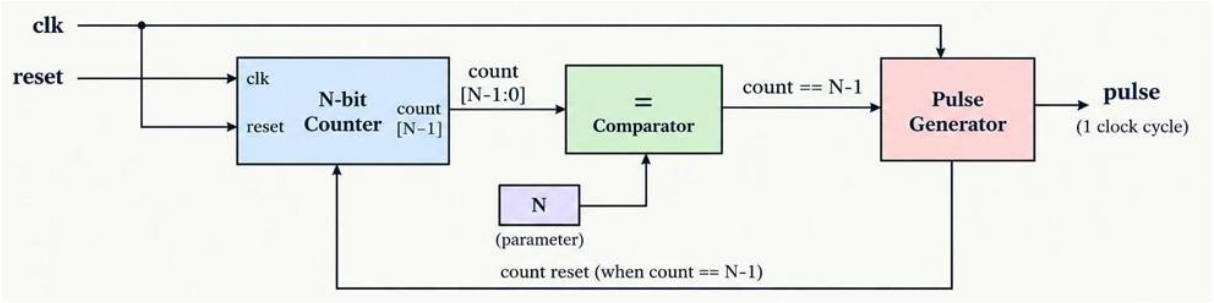
10. Divide-by-N Counter

Aim: To design a divide-by-N counter using Verilog HDL.

Theory:A divide-by-N counter is a sequential circuit that divides the input clock frequency by a selected factor N. It counts clock pulses and generates an output signal when the specified count is reached. It is commonly used in digital systems for clock division and frequency control.

Working: The counter increments on every active clock edge until it reaches the programmed terminal count. When the terminal count is reached, the counter resets or the output toggles, depending on the implementation. This process repeats continuously to produce an output signal with a lower frequency than the input clock.

Block diagram

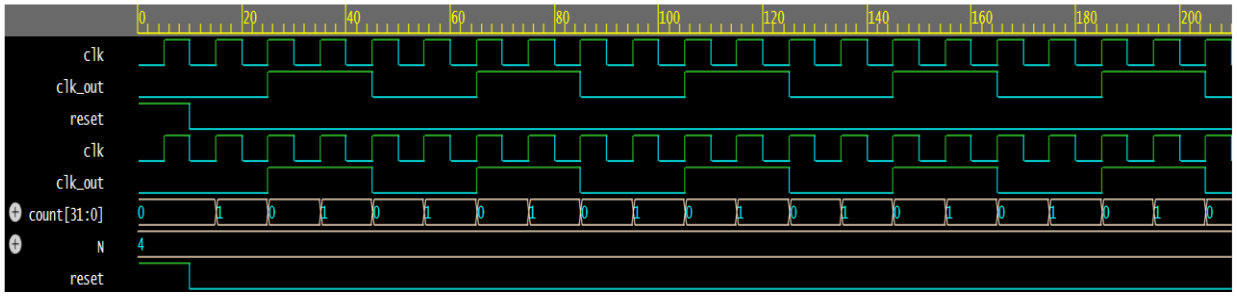


Code

```verilog
// Testbench
module tb_divide_by_n;
reg clk, reset;
wire clk_out;
divide_by_n #(4) uut (
    .clk(clk),
    .reset(reset),
    .clk_out(clk_out)
);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    clk = 0;
    reset = 1;
    #10 reset = 0;
    #200;
    $finish;
end
initial begin
    $monitor("Time=%0t Reset=%b Clock_Out=%b",
             $time, reset, clk_out);
end
endmodule
```

```verilog
// Design: Divide-by-N Counter
module divide_by_n #(parameter N = 4)(
    input clk,
    input reset,
    output reg clk_out
);
integer count;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        count <= 0;
        clk_out <= 0;
    end
    else begin
        if (count == (N/2)-1) begin
            count <= 0;
            clk_out <= ~clk_out;
        end
        else begin
            count <= count + 1;
        end
    end
end
endmodule
```

Output



11. Mealy and Moore Finite State Machines (FSMs)

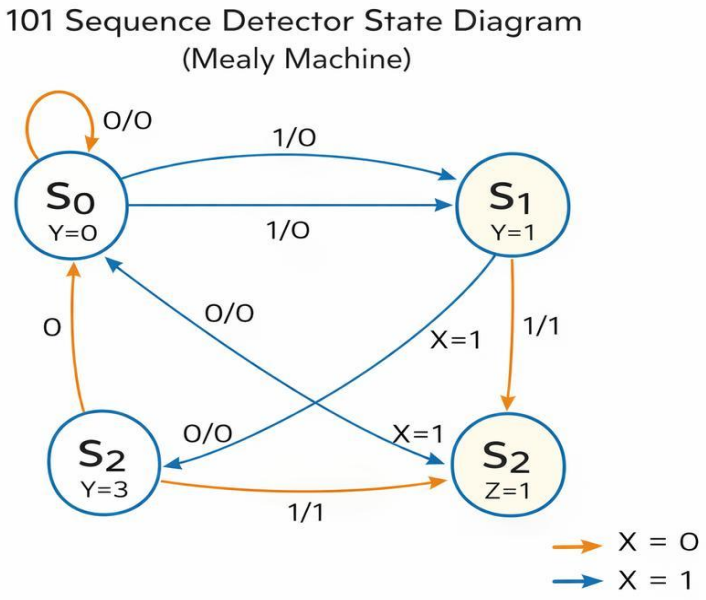
A. Mealy FSM — Sequence Detector for 101

Aim: To design a Mealy FSM that detects the input sequence 101.

Theory: A Mealy finite state machine produces its output from both the current state and the current input. In a sequence detector, states represent how much of the target sequence has been matched.

Working: For the sequence 101, the FSM updates its state as each input bit arrives. The output becomes active when the current state and input complete the sequence. Overlapping detection depends on the transition design.

Design



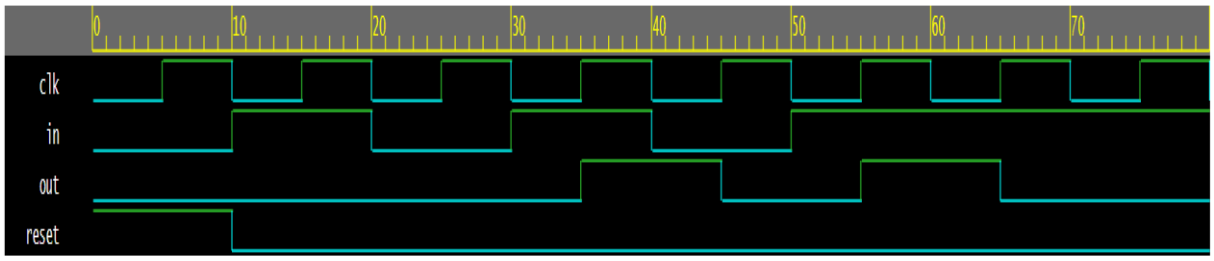
Code

```
// Testbench
module tb_mealy_fsm;
reg clk, reset, in;
wire out;
mealy_fsm uut (
    .clk(clk),
    .reset(reset),
    .in(in),
    .out(out)
);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;

    clk = 0;
    reset = 1;
    in = 0;
    #10 reset = 0;
    in = 1; #10;
    in = 0; #10;
    in = 1; #10;
    in = 0; #10;
    in = 1; #10;
    #20;
    $finish;
end
initial begin
    $monitor("Time=%0t in=%b out=%b",
             $time, in, out);
end
endmodule
```

```
// Design: Mealy Sequence Detector for 101
module mealy_fsm(
    input clk,
    input reset,
    input in,
    output reg out
);
reg [1:0] state;
parameter S0 = 2'b00,
          S1 = 2'b01,
          S2 = 2'b10;
always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= S0;
        out <= 1'b0;
    end
    else begin
        case (state)
            S0: begin
                out <= 1'b0;
                if (in)
                    state <= S1;
                else
                    state <= S0;
            end
            S1: begin
                out <= 1'b0;
                if (in)
                    state <= S1;
                else
                    state <= S2;
            end
            S2: begin
                if (in) begin
                    out <= 1'b1;
                    state <= S1;
                end
                else begin
                    out <= 1'b0;
                    state <= S0;
                end
            end
            default: begin
                state <= S0;
                out <= 1'b0;
            end
        endcase
    end
end
endmodule
```

Output



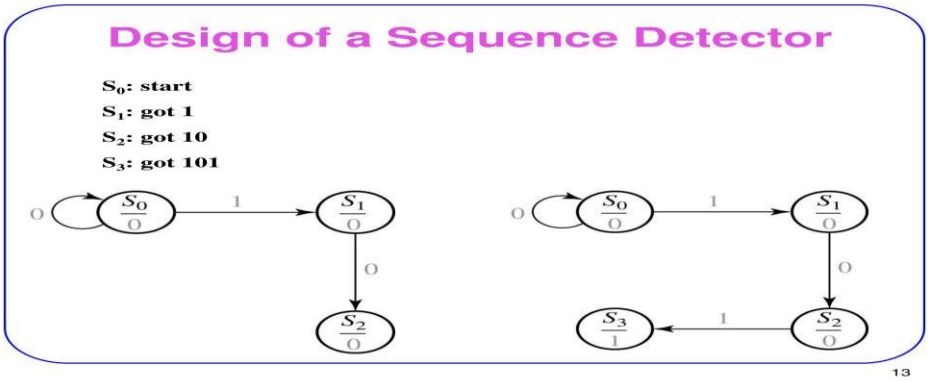
B. Moore FSM — Sequence Detector for 101

Aim: To design a Moore FSM that detects the input sequence 101.

Theory: A Moore finite state machine produces its output from the current state only. A separate detection state is generally used to indicate that the required input sequence has been recognized.

Working: For the sequence 101, the FSM changes state as input bits arrive. When the final 1 completes the sequence, the machine enters the detection state and asserts the output. The output remains associated with that state.

Design



Code

```
// Testbench
module tb_moore_fsm;
reg clk, reset, in;
wire out;
moore_fsm uut (
    .clk(clk),
    .reset(reset),
    .in(in),
    .out(out)
);

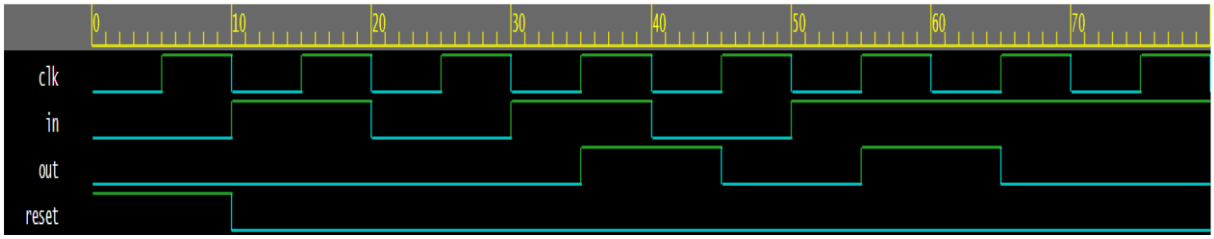
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    clk = 0;
    reset = 1;
    in = 0;
    #10 reset = 0;
    in = 1; #10;
    in = 0; #10;
    in = 1; #10;
    in = 0; #10;

    in = 1; #10;
    #20;
    $finish;
end
initial begin
    $monitor("Time=%0t in=%b out=%b",
             $time, in, out);
end
endmodule
```

```
// Design: Moore Sequence Detector for 101
module moore_fsm(
    input clk,
    input reset,
    input in,
    output reg out
);
reg [1:0] state;
parameter S0 = 2'b00,
          S1 = 2'b01,
          S2 = 2'b10,
          S3 = 2'b11;
always @(posedge clk or posedge reset) begin
    if (reset)
        state <= S0;
    else begin
        case (state)
            S0: state <= in ? S1 : S0;
            S1: state <= in ? S1 : S2;
            S2: state <= in ? S3 : S0;

            S3: state <= in ? S1 : S2;
            default: state <= S0;
        endcase
    end
end
always @(*) begin
    case (state)
        S3: out = 1'b1;
        default: out = 1'b0;
    endcase
end
endmodule
```

Output



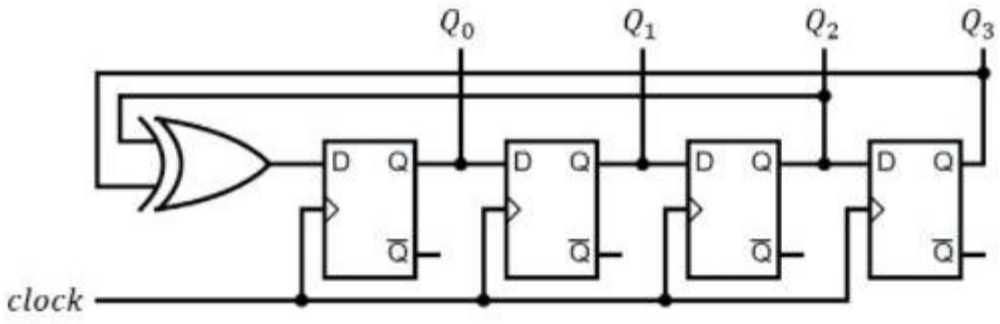
12. Random Sequence Generator (4-Bit LFSR)

Aim: To design a 4-bit linear feedback shift register (LFSR) for generating a pseudo-random binary sequence.

Theory: A 4-bit Linear Feedback Shift Register (LFSR) generates a pseudo-random binary sequence using flip-flops and XOR feedback.

Working: At each clock edge, the register shifts its bits, and the XOR-generated feedback bit enters the register to produce the next sequence.

Block diagram



Code

```
// Testbench
module tb_random_sequence_generator;
reg clk, reset;
wire [3:0] q;
random_sequence_generator uut (
    .clk(clk),
    .reset(reset),
    .q(q)

);
always #5 clk = ~clk;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars;
    clk = 0;
    reset = 1;

    #10 reset = 0;
    #200;
    $finish;
end
initial begin
    $monitor("Time=%0t q=%b", $time, q);
end
endmodule
```

```
// Design: 4-bit LFSR
module random_sequence_generator(
    input clk,
    input reset,
    output reg [3:0] q
);
wire feedback;
assign feedback = q[3] ^ q[2];
always @(posedge clk or posedge reset) begin
    if (reset)
        q <= 4'b0001;
    else
        q <= {q[2:0], feedback};
end
endmodule
```

Output

